Search Feature + Role management Performance Docs.

This purpose of this performance testing document is to record the application response time and DB query performance of the Curriculum Mapping tool after adding the tool's end-to-end search feature + role access management restrictions on said search feature. Performance testing will emphasize testing requests and queries that relate to the search feature functionality. This document will provide an in-depth overview of the search feature performance, and a high-level overview of the entire app's performance.

Testing DB State:

There is now an optional seeder that populates the DB with realistic entities and data size. The seeder aims to use generic names with increasing indices, while aiming to have around a couple thousand entries per entity (However, custom commands can be used to override the default size the seeder creates, see below).

All performance testing is done using the DB-state that this seeder builds.

Seeder Summary:

- **File path:** laravel/database/seeders/SearchPerformanceSeeder.php
- **How to run default seeder:**
  From the Laravel Directory, run "php artisan db:seed --class=SearchPerformanceSeeder"
- **How to provide custom DB size:**
  Run the following command from the laravel directory to modify the number of courses and prorgrams created (change the variable values):

"SEARCH_PERFORMANCE_COURSES=5000 SEARCH_PERFORMANCE_PROGRAMS=2500 php artisan db:seed --class=SearchPerformanceSeeder"

- **Default DB state – the default seeder run creates:**
  - 2,000 courses
  - 1,500 programs
  - 4,000 course_programs relationships
  - 2,000 course descriptions, 1 per course
  - 14,000 course topics, 7 per course
  - 12,000 learning outcomes, 6 per course
  - 8,000 assessment methods, 4 per course

- 4,000 course materials, 2 per course
- 2,100 direct course_users rows
- Approximately 4,000 materialized course_user_role rows
- 1,600 direct program_users rows
- Approximately 2,500 program_user_role rows
- One faculty containing two departments
- 100 reusable course codes
- Course levels ranging from 100 to 699

- **Testing accounts:**
  - search-performance-no-access@example.test: 0 accessible courses
  - search-performance-direct@example.test: 100 directly accessible courses and 100 direct program rows
  - search-performance-owner@example.test: all 2,000 courses and 1,500 programs through direct access
  - search-performance-program-director@example.test: 877 distinct courses through 300 directed programs
  - search-performance-department-head@example.test: 1,000 courses from Performance Test Department A and 750 program role rows
  - search-performance-faculty-wide@example.test: all 2,000 faculty courses and 1,500 programs through materialized role access
  - search-performance-admin@example.test: all system courses through the administrator bypass without requiring access rows
  - The default password for all testing accounts is "password"
- **Repeatable queries – main benchmark terms:**
  - climate: common query
  - watershed: medium-sized query
  - cryosphere: rare query
  - PFAA100: exact course-code query

  Expected course counts for the default dataset:

Account	Climate	Watershed	Cryosphere
No access	0	0	0
Limited direct	50	30	0
Broad direct owner	1,000	599	1
Program Director	439	264	1
Department Head	500	299	1
Faculty-wide	1,000	599	1
Administrator	1,000	599	1

These expected counts only cover generated performance courses. Administrators may receive additional matches if the database already contains other matching courses.

- **Safety and reuse**
  - The seeder refuses to run in production.
  - Inserts are processed in chunks of 500 rows.
  - All work runs inside a database transaction.
  - Rerunning removes and rebuilds only its own Performance Test data.
  - Existing application data is not deleted.
  - PostgreSQL sequences are synchronized before inserting.
  - Generated tsvector values are created automatically.
  - Generic indexed names make the dataset deterministic.
  - Search terms are balanced across departments and access types.

  The seeder has been rerun without duplication, its search vectors and access distributions were verified, and all 68 Course Search tests continue to pass.

  The number of programs shown in search filters can be higher than the number of direct program-access rows because search includes programs containing at least one accessible course.

Testing Environment

The following is the recorded environment at the time of testing:

- **Testing date:** August 11-12, 2026
- **Branch:** Kabir/search-role-access-performance-testing
- **Commit:** 32c133c
- **Machine:** MacBook Air, Apple M1, 8-core CPU, 8 GB RAM
- **Operating system:** macOS 15.6.1
- **PHP:** 8.4.20
- **PostgreSQL:** 18.3

Performance Testing Plan

The goal is to evaluate the current DB-side search access implementation using application response times and PostgreSQL query cost/execution time. Search access is filtered using course_users and course_user_role, while administrators bypass access filtering.

1. Create a local performance seeder with generic indexed data:
2. Create benchmark users with:
   a. No access.
   b. Limited direct access.
   c. Limited, medium, and broad elevated access.

    d. Administrator access.
3. Confirm access correctness before measuring performance.
4. Test a repeatable set of searches:
    a. Exact course codes.
    b. Rare and common terms.
    c. AND and OR queries.
    d. Property, course-code, level, and program filters.
    e. Course View and Program View.
    f. Different access roles and access sizes.
5. Run 3–5 warm-up requests followed by approximately 20 measured requests per scenario. Record median, p95, maximum response time, and result count.
    a. Only record the non-warm-up requests.
6. Use EXPLAIN (ANALYZE, BUFFERS) for slow queries. Check index usage, sequential scans, nested loops, rows filtered, execution time, and buffer activity.

Performance Results: 2,000-Course Dataset

The first round of application-side performance testing is complete. Tests were performed against the authenticated /search route using the database state created by SearchPerformanceSeeder.

Each account ran the same six queries in Course View and Program View:

• PFAA100: exact course-code query

• cryosphere: rare query

• watershed: medium-sized query

• climate: common query

• climate AND watershed: query requiring both terms

• climate OR watershed: query allowing either term

For each scenario, five warm-up requests were discarded before running 20 measured requests at concurrency 1. The rendered result counts were confirmed before each benchmark.

In total:

• 88 scenarios were measured

• 440 warm-up requests were performed

• 1,760 measured requests were performed

• All requests returned an authenticated HTTP 200 response

• No failed or non-200 requests occurred

• No Laravel errors occurred during testing

The values below are shown as:

Mean / Median / P95 response time in milliseconds

Because each scenario contained 20 measured requests, ApacheBench's P95 value is also the maximum observed response time.

Course View Response Times

Query	No Access	Direct	Program Director	Department Head	Owner	Faculty-Wide	Admin
PFAA100	100 / 101 / 106	113 / 112 / 127	137 / 136 / 183	162 / 159 / 222	200 / 196 / 247	197 / 196 / 250	119 / 116 / 160
cryosphere	100 / 100 / 105	113 / 111 / 159	132 / 132 / 137	160 / 161 / 167	194 / 194 / 200	195 / 195 / 203	116 / 116 / 121
watershed	110 / 107 / 152	129 / 129 / 137	197 / 197 / 206	215 / 216 / 228	360 / 355 / 414	362 / 359 / 396	238 / 238 / 243
climate	125 / 124 / 134	172 / 172 / 181	408 / 383 / 738	455 / 454 / 465	794 / 785 / 864	787 / 788 / 813	650 / 641 / 730
climate AND watershed	102 / 101 / 114	114 / 115 / 119	139 / 138 / 150	160 / 160 / 165	220 / 220 / 227	223 / 219 / 278	137 / 137 / 143
climate OR watershed	130 / 130 / 139	175 / 176 / 186	401 / 403 / 418	464 / 464 / 470	881 / 834 / 1,702	845 / 840 / 950	696 / 694 / 744

<u>Program View Response Times</u>

Query	No Access	Direct	Program Director	Department Head	Owner	Faculty-Wide	Admin
PFAA100	104 / 103 / 109	115 / 115 / 119	135 / 135 / 143	164 / 164 / 169	202 / 199 / 241	199 / 198 / 204	119 / 119 / 128
cryosphere	104 / 104 / 110	114 / 114 / 117	135 / 135 / 140	166 / 166 / 173	199 / 199 / 205	200 / 201 / 210	118 / 119 / 125
watershed	119 / 119 / 124	142 / 142 / 152	219 / 217 / 234	241 / 239 / 261	390 / 386 / 475	388 / 384 / 441	264 / 263 / 277
climate	139 / 137 / 189	191 / 185 / 249	419 / 418 / 437	482 / 482 / 493	834 / 827 / 976	831 / 818 / 1,030	682 / 680 / 732
climate AND watershed	106 / 106 / 115	120 / 120 / 133	151 / 153 / 156	162 / 163 / 172	227 / 227 / 235	235 / 230 / 302	148 / 143 / 207

climate OR watershed	175 / 139 / 566	190 / 188 / 206	450 / 451 / 458	503 / 503 / 514	877 / 874 / 950	886 / 885 / 921	809 / 737 / 1,920

Additional Admin Results

Admin could access 2,097 total system courses and 1,553 programs. This includes the generated performance data and records that already existed in the local database.

View	Query	Results	Mean	Median	P95
Courses	forest	1,017 / 1,511	640 ms	637 ms	700 ms
Courses	climate -watershed	1,029 / 1,513	644 ms	641 ms	684 ms
Programs	forest	1,017 / 1,511	684 ms	673 ms	769 ms
Programs	climate -watershed	1,028 / 1,513	681 ms	678 ms	727 ms

Main Findings

• Response time generally increased as the number of accessible results increased.

• Empty and rare searches normally completed in approximately 100–200 ms.

• Searches returning around 1,000 courses and 1,500 programs generally completed in approximately 640–886 ms.

• Program View was usually slightly slower than Course View because it groups matching courses by program and may process additional direct program-name matches.

• Owner and faculty-wide performance was very similar when both accounts returned the same results. This suggests direct and materialized full-access rows have comparable application-side performance for this dataset.

• Admin was faster than Owner and faculty-wide for broad searches, even though Admin could see slightly more data. This is likely related to Admin bypassing the access-row restrictions.

FIlter combinations response times:

Filter Performance Testing

A separate Admin benchmark was completed to measure performance with different filter combinations. Every scenario used the climate query and was tested in both Course View and Program View.

Each scenario included five warm-up requests followed by 20 measured requests at concurrency 1. The selected filters and result counts were confirmed from the rendered page before testing.

View	Filters	Results (Courses/Programs)	Mean	Median	P95
Courses	Topics only	1,024 / 1,513	345 ms	345 ms	361 ms
Courses	Course code: PFAB	20 / 30	142 ms	139 ms	202 ms
Courses	Course level: 300	157 / 308	215 ms	215 ms	222 ms
Courses	Program 0204	2 / 2	123 ms	123 ms	128 ms
Courses	Topics + PFAD + 300 level + Program 0204	1 / 2	107 ms	108 ms	114 ms
Programs	Topics only	1,023 / 1,513	386 ms	383 ms	450 ms
Programs	Course code: PFAB	20 / 30	149 ms	149 ms	156 ms
Programs	Course level: 300	157 / 365	238 ms	232 ms	327 ms
Programs	Program 0204	2 / 1	130 ms	130 ms	136 ms
Programs	Topics + PFAD + 300 level + Program 0204	1 / 1	116 ms	115 ms	152 ms

All 200 measured requests returned authenticated HTTP 200 responses with no failed requests or Laravel errors. Selective filters completed in approximately 107–238 ms. The topics-only filter still returned over 1,000 courses, but reduced the Admin all-properties response time from approximately 650–682 ms to 345–386 ms. Overall, filter performance was good for the current 2,000-course dataset.

<u>Additional Notes:</u>

- Despite using different access-control modalities, the "Owner" and "Faculty-wide" cases performed very similarly, suggesting their individual access permission check procedures perform similarly and the main influencing factor is the size of the result set
- The "Admin" was case consistently faster than "Owner" and "Faculty-wide" likely due to the lack of need for checking access permissions

- For the course view tests, one the 'OR' query results stood out to me. The P95 for the "Owner" case was much higher than that of the "Faculty-wide" case despite having access to the same courses (perhaps the influence of some external factors while running those tests?)
- For the program view tests, the 'OR' query also produced an interesting result. The P95 for the "Admin" case was over double that of the "Faculty-wide" and "Owner" cases

<u>P95 Response-Time Recheck</u>

Two unusually high P95 results were rechecked using the same accounts, queries, views, URLs, five warm-up requests, and concurrency level of one. The measured sample was increased from 20 to 100 requests to reduce the effect of isolated slow responses.

Scenario	Original P95	Rechecked P95	Median	Maximum	Failed Requests
Owner, Course View, climate OR watershed	1,702 ms	878 ms	838 ms	2,005 ms	0
Admin, Program View, climate OR watershed	1,920 ms	742 ms	735 ms	778 ms	0

During the Owner recheck, one request still took 2,005 ms, but 95% of requests completed within 878 ms. This result is close to the original Faculty-wide P95 of 950 ms, which is reasonable because both accounts can access the same generated courses.

The Admin spike did not repeat. Its rechecked P95 was 742 ms with a maximum of 778 ms. This was faster than the Owner and Faculty-wide cases, which is consistent with administrators bypassing access-row filtering.

Overall, the rechecks suggest that the original high P95 values were isolated timing spikes rather than consistent performance problems. Both scenarios completed all 100 requests without failures.